

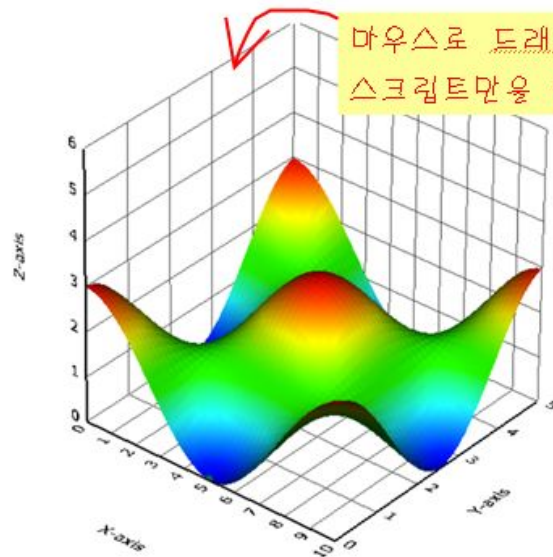
CHAPTER 9.

자바 스크립트 기초



자바 스크립트 소개

- 자바스크립트(javascript): 동적인 웹 페이지를 작성하기 위하여 사용되는 언어
- 웹의 표준 프로그래밍 언어
- 모든 웹브라우저들은 자바스크립트를 지원



마우스로 드래그하여 회전할 수 있는 이러한 화면도 자바 스크립트만을 이용하여 구현되었다.

HTML5 기술의 핵심



자바 vs 자바 스크립트

특징	자바 언어	자바스크립트
언어 종류	소스 파일을 컴파일하여 실행하는 컴파일 언어이다.	브라우저가 소스 코드를 직접 해석하여 실행하는 인터프리트 언어이다.
실행 방식	자바 가상 기계 위에서 실행한다.	브라우저 위에서 실행된다.
작성 위치	별도의 소스 파일에 작성	HTML 파일 안에 삽입 가능
변수 선언	변수의 타입을 반드시 선언해야 함	변수의 타입을 선언하지 않아도 사용 가능

자바 스크립트 역사

- 넷스케이프의 브렌던 아이크(Brendan Eich)가 개발
- 처음에는 라이브스크립트(LiveScript)
- 최신 버전은 자바스크립트 1.8.5
- ECMA(European Computer Manufacturer's Association)이 ECMAScript라는 이름으로 표준을 제정-> ECMA-262



자바 스크립트 특징

- 인터프리트 언어
- 동적 타이핑(dynamic typing)
- 구조적 프로그래밍 지원
- 객체 기반
- 함수형 프로그래밍 지원
- 프로토타입-기반(prototype-based)

자바 스크립트의 용도

- 이벤트에 반응하는 동작을 구현
- **AJAX** : 서버와의 **HTTP** 통신을 비동기 방식으로 수행하는 기술
- **HTML** 요소들의 콘텐츠 및 크기나 색상을 동적으로 변경
- 게임이나 애니메이션
- 사용자가 입력한 값들을 검증

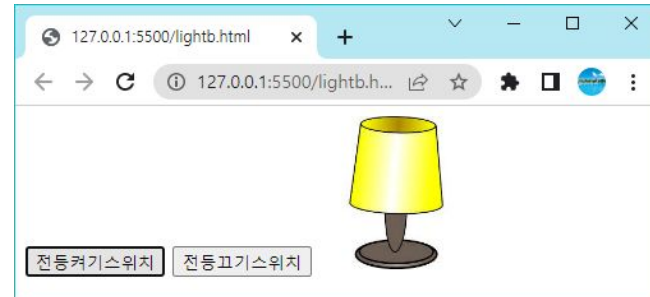
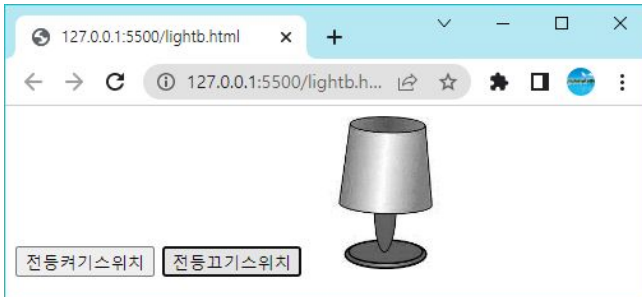
자바스크립트로 할 수 있는 일 1.

- 버튼 클릭 이벤트를 통해 이미지 변경하기

```
<!DOCTYPE html>
<html>
<body>

<button
onclick="document.getElementById('lightb').src='light-bulb1.png'">전등켜기스위치 </button>
<button
onclick="document.getElementById('lightb').src='light-bulb2.png'">전등끄기스위치 </button>

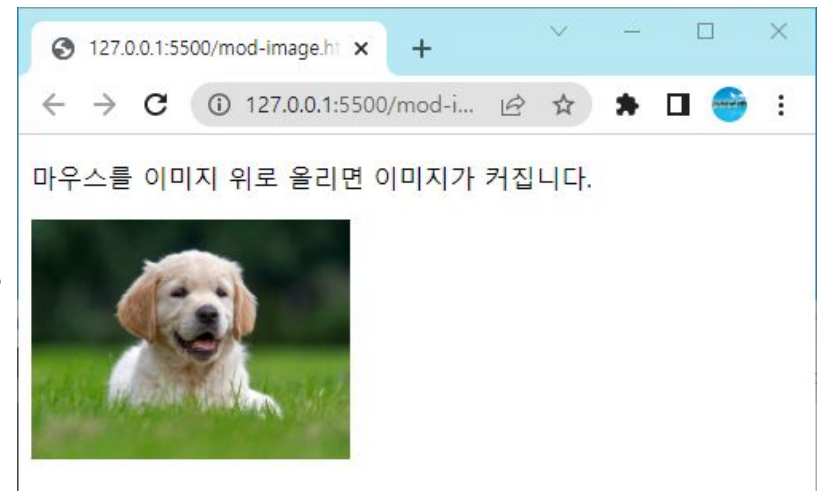
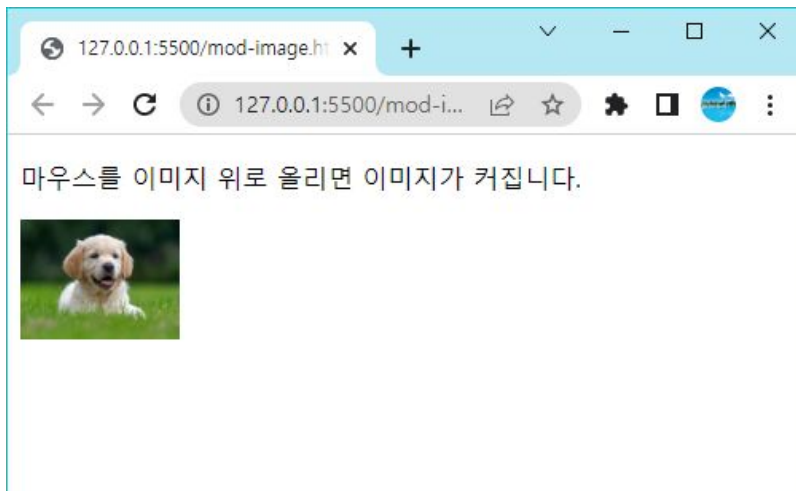

</body>
</html>
```



자바스크립트로 할 수 있는 일 2.

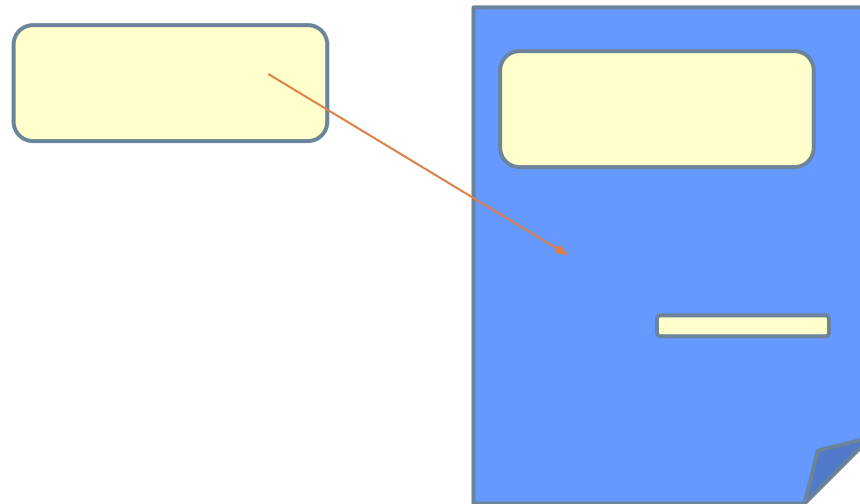
- 이미지 위에 마우스 커서가 위치했을 때 이미지를 커지게 하기

```
<!DOCTYPE html>
<html>
<body>
  <p>마우스를 이미지 위로 올리면 이미지가 커집니다.</p>
  
</body>
</html>
```



자바 스크립트의 위치

- CSS와 마찬가지로 자바스크립트도 다음과 같은 3가지 방법으로 HTML 문서에 삽입될 수 있다.
 - 내부 자바스크립트
 - 외부 자바스크립트
 - 인라인 자바스크립트



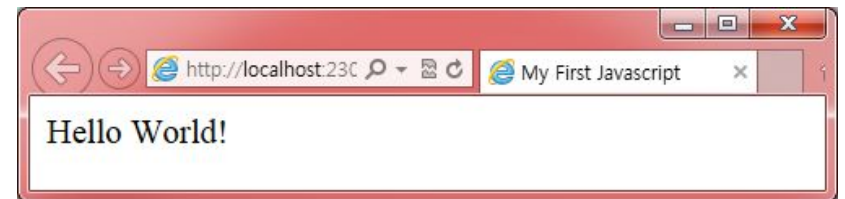
내부 자바 스크립트

```
<!DOCTYPE HTML>
<html>
<head>

  <title>My First Javascript </title>

  <script>
    document.write("Hello World!");
  </script>

</head>
<body></body>
</html>
```

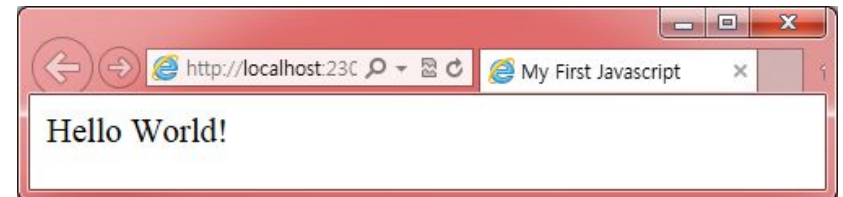


외부 자바 스크립트

```
<!DOCTYPE html>
<html>
<head>
  <script src="myscript.js"></script>
</head>
<body>
</body>
</html>
```

myscript.js

```
document.write("Hello World!");
```



인라인 자바 스크립트

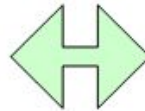
```
<!DOCTYPE html>  
<html>  
<body>  
  <button type="button" onclick="alert('반갑습니다.')">버튼을 누르세요!</button>  
</body>  
</html>
```



문장(명령, 실행문)

- 자바스크립트 문장(statement)들은 웹 브라우저에게 내리는 명령

1. 문서에 "Hello World!"를 추가하시오.
2. 화면에 경고창을 띄우시오.
3. 변수를 하나 만드시오.
4. ...



```
document.write("Hello World!");  
alert("warning!!");  
var count;  
...
```

↓
순차적
으로
실행된
다.

주석

처음 나왔죠? 웹 페이지에서 특정한 아이디가 붙여진
요소를 찾아서 그 콘텐츠를 변경하는 문장입니다.

```
// id가 heading1인 heading요소를 찾아서 내용을 바꾼다.
```

```
document.getElementById("heading1").innerHTML = "My HomePage";
```

```
/*
```

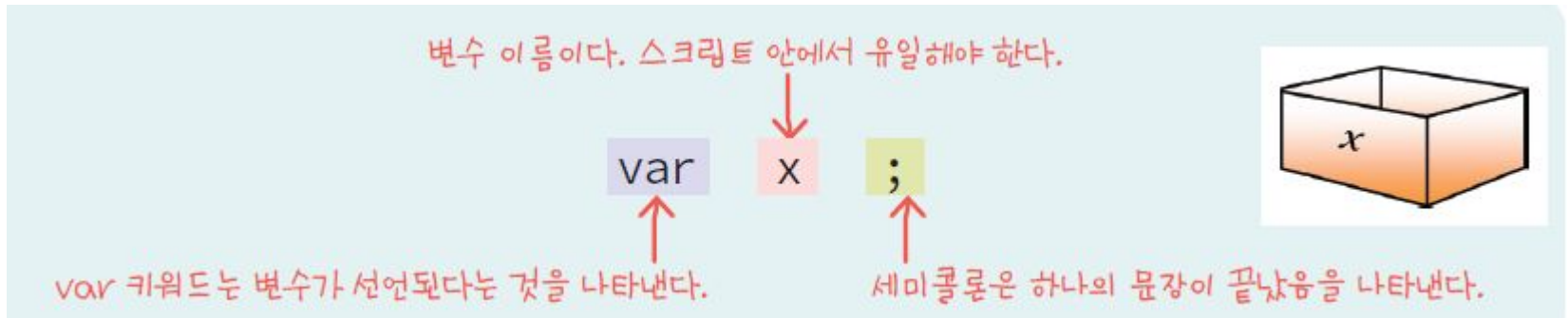
```
    이 코드는 웹 페이지의 heading의 내용을 변경한다.
```

```
*/
```

```
document.getElementById("heading1").innerHTML = "My HomePage";
```

변수

- 변수(**variable**)는 데이터를 저장하는 상자
- `var` 키워드를 사용하여 선언(**declare**)한다.

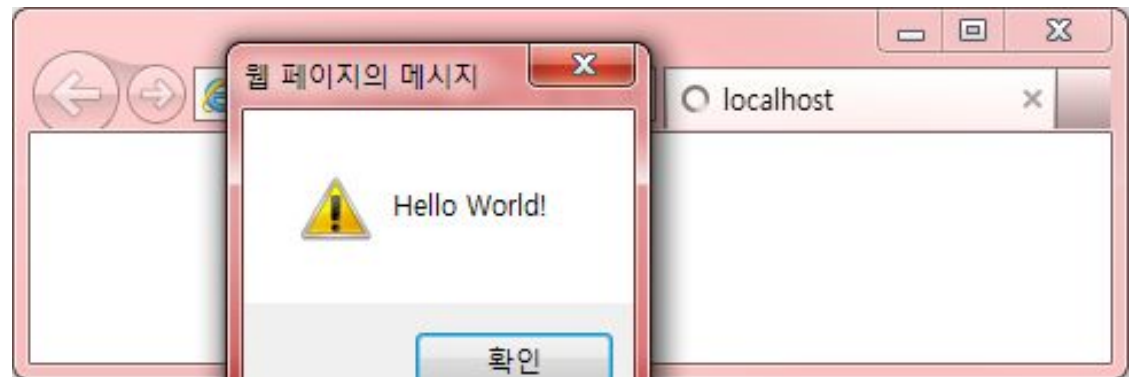


새로운 JavaScript 문법 [버전 ECMAScript2015 (ES6)]

- **let** 키워드는 2015년에 자바스크립트에 추가되었다. `var`에 비하여 장점이 있어서 사용이 권장된다.
- **const** 키워드는 상수(변하지 않는 변수)를 선언하기 위하여 도입되었다. 개발자가 변경시킬 필요가 없는 객체는 `const`로 정의하는 것이 권장된다.

예제

```
<script>  
  var x;  
  x = "Hello World!";  
  alert(x);  
</script>
```



자료형

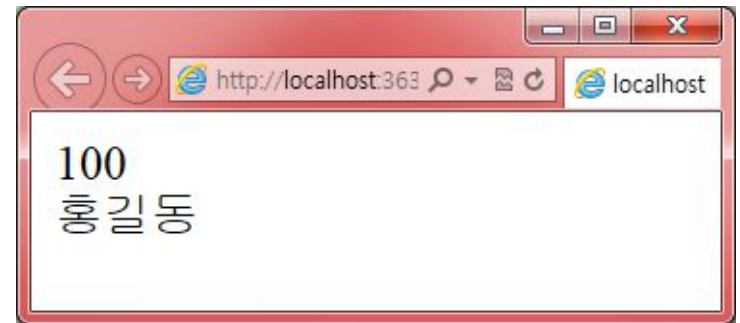
- 수치형(number)
- 문자열(string)
- 부울형(boolean)
- 객체형(object)
- undefined

예 제

```
<script>
  var s;

  s = 100;
  document.write(s + "<br>");

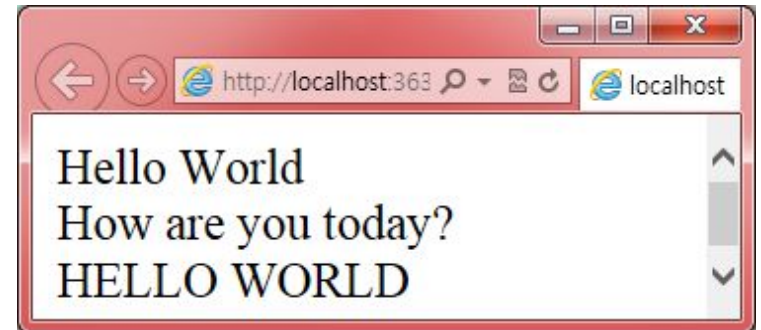
  s = "홍길동";
  document.write(s + "<br>");
</script>
```



예제

```
<script>
  var s = "Hello World";
  var t = "How are you" + " today?";

  document.write(s + "<br>");
  document.write(t + "<br>");
  document.write(s.toUpperCase() + "<br>");
</script>
```



템플릿 리터럴

- 자바스크립트에서 문자열 안에 변수값을 넣으려면 ``...${ expression }...`` 형태의 문자열을 사용하는 것이 좋다.
- 이 백틱기호(``...``)를 사용한 문자열은 **자바스크립트 ES6 버전에서 추가**된 것으로 템플릿 리터럴(template literal)이라고 부른다.

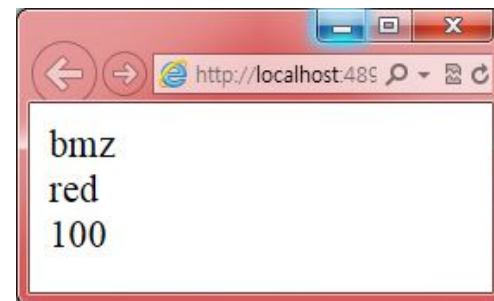
``Hello ${x}번``



변수 `x`의 값을 문자열 안에 넣는다.
만약 `x`가 10이라면 “Hello 10번”이 된다.

객체형

- 객체(**object**)는 사물의 속성과 동작을 묶어서 표현하는 기법
- (예) 자동차는 메이커, 모델, 색상, 마력과 같은 속성도 있고 출발하기, 정지하기 등의 동작도 가지고 있다.
- `var myCar = { model: "bmz", color: "red", hp: 100 };`
- `document.write(myCar.model + "<br>");`
- `document.write(myCar.color + "<br>");`
- `document.write(myCar.hp + "<br>");`

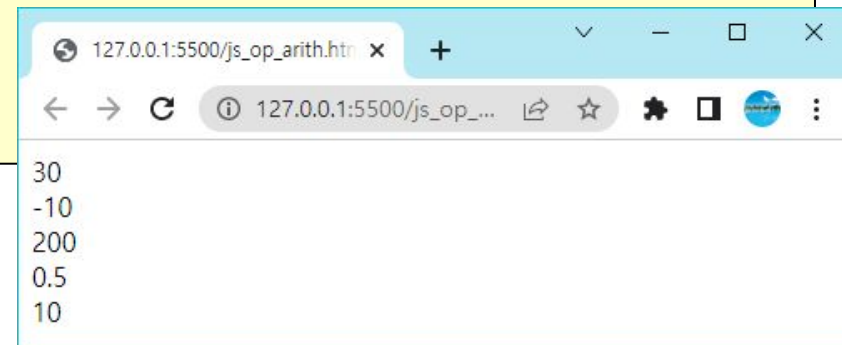


연산자

연산자	설명	예	수식의 값
+	덧셈	$x=3+2$	5
-	뺄셈	$x=3-2$	1
*	곱셈	$x=3*2$	6
/	나눗셈	$x=3/2$	1.5
%	나머지	$x=3\%2$	1
++	증가	$++x$	x의 값 3 → 4
--	감소	$--x$	x의 값 3 → 2

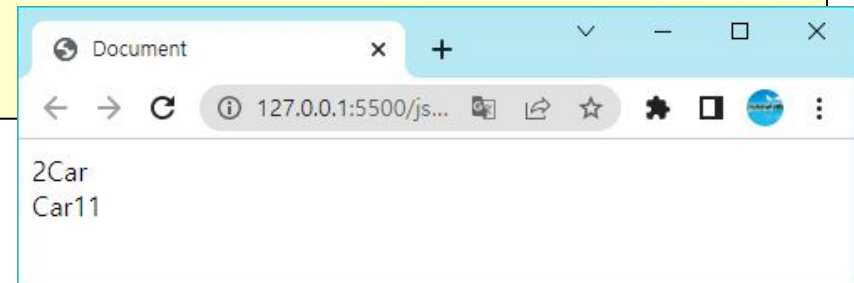
예제

```
...  
<script>  
  var x = 10;  
  var y = 20;  
  
  document.write(x+y+"<br>");  
  document.write(x-y+"<br>");  
  document.write(x*y+"<br>");  
  document.write(x/y+"<br>");  
  document.write(x%y+"<br>");  
</script>
```



문자열에서의 + 연산자

```
<!DOCTYPE html>
<body>
  <script>
    var x, y;
    x = 1 + 1 + "Car";
    y = "Car" + 1 + 1;
    document.write(x + "<br>");
    document.write(y + "<br>");
  </script>
</body>
</html>
```



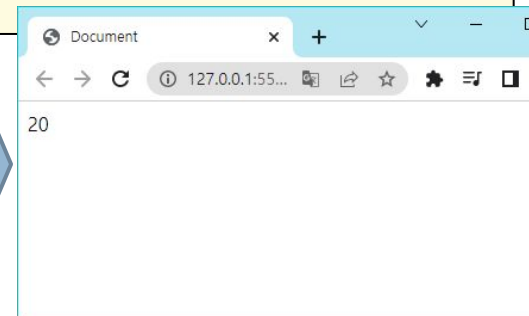
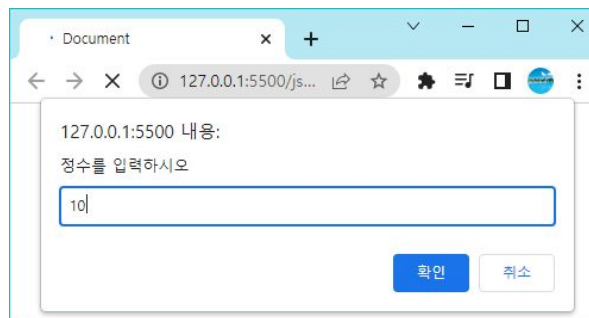
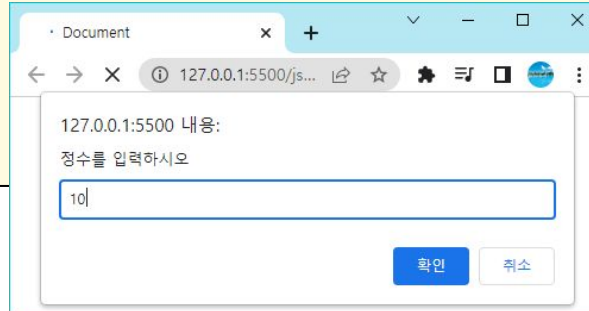
예제

```
<script>
  var x, y;
  var input;

  input = prompt("정수를 입력하시오", "정수로");
  x = parseInt(input);

  input = prompt("정수를 입력하시오", "정수로");
  y = parseInt(input);

  document.write(x + y + "<br>");
</script>
```



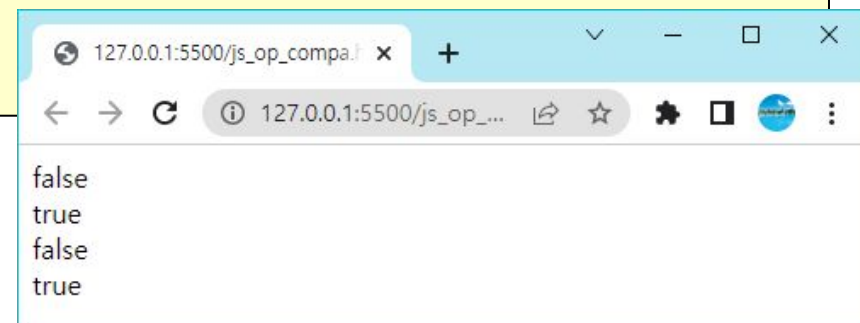
비교 연산자

연산자	설명	예	결과값
==	값이 같으면 참	x==1	true
		x==2	false
!=	값이 다르면 참	x!=2	true
>	크면 참	x>2	false
<	작으면 참	x<2	true
>=	크거나 같으면 참	x>=2	false
<=	작거나 같으면 참	x<=2	true

예제

```
<script>
  var x = 10;
  var y = 20;

  document.write((x > y) + "<br>");
  document.write((x < y) + "<br>");
  document.write((x == y) + "<br>");
  document.write((x != y) + "<br>");
</script>
```



=== 연산자와 !== 연산자

- === 연산자는 피연산자의 값과 데이터 타입이 모두 일치할 때만 참을 반환한다. !== 연산자는 값이 다르거나 데이터 타입이 다르면 참이 된다.

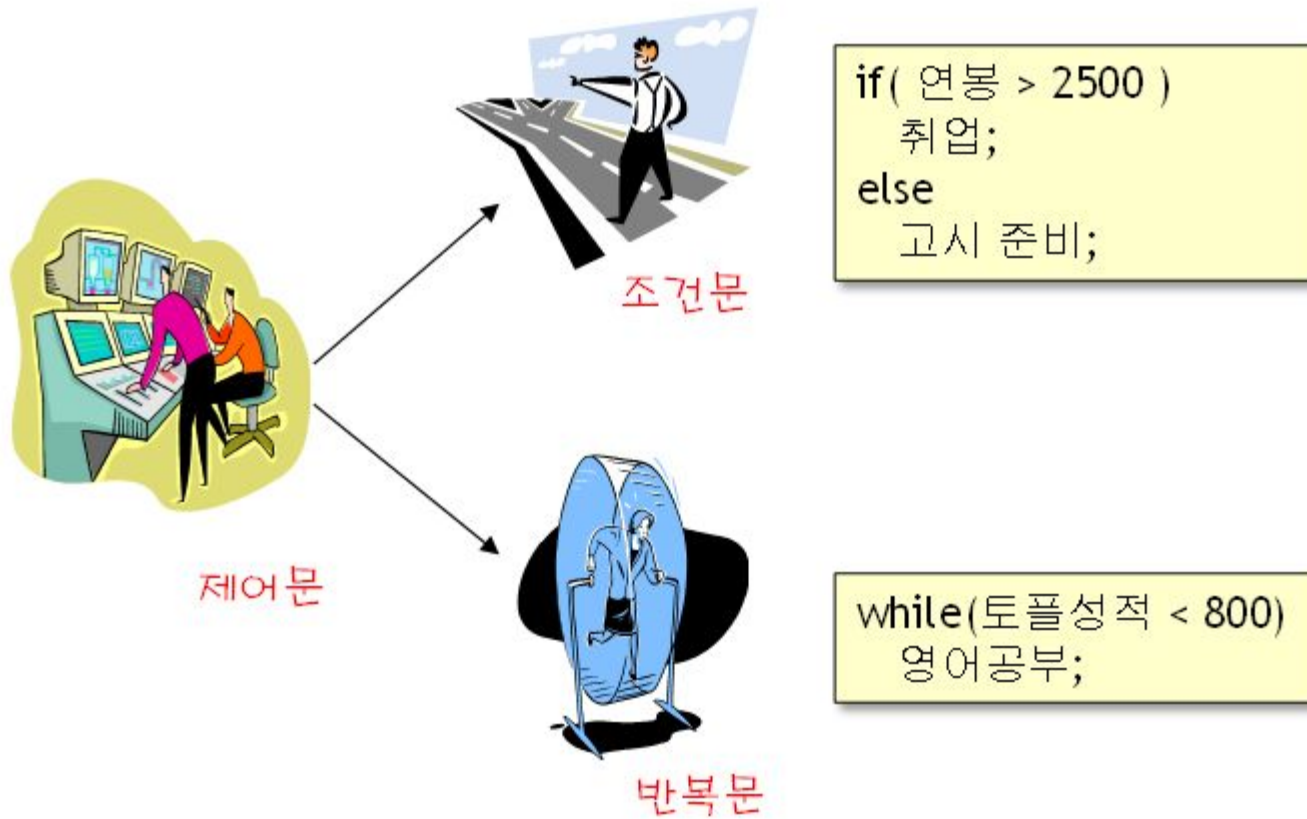
연산자	설명	예	결과값
===	값과 타입이 모두 같으면 참	x===1	true(값도 같고 타입도 같다)
		x==="1"	false(값은 같으나 타입이 다르다)
!==	값이나 타입이 다르면 참	x!==1	false(값도 같고 타입도 같다)
		x!== "1"	true(값은 같으나 타입이 다르다)

논리 연산자

- 논리 연산자는 여러 개의 조건을 조합하여 참인지 거짓인지를 따질 때 사용한다.

연산자 기호	사용 예	의미
&&	x && y	AND 연산, x와 y가 모두 참이면 참, 그렇지 않으면 거짓
	x y	OR 연산, x나 y중에서 하나만 참이면 참, 모두 거짓이면 거짓
!	!x	NOT 연산, x가 참이면 거짓, x가 거짓이면 참

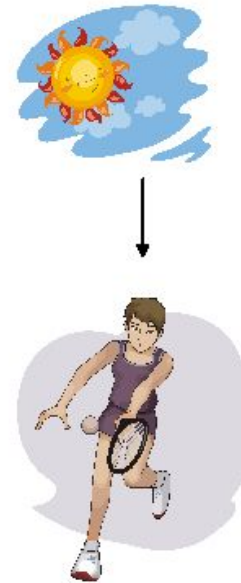
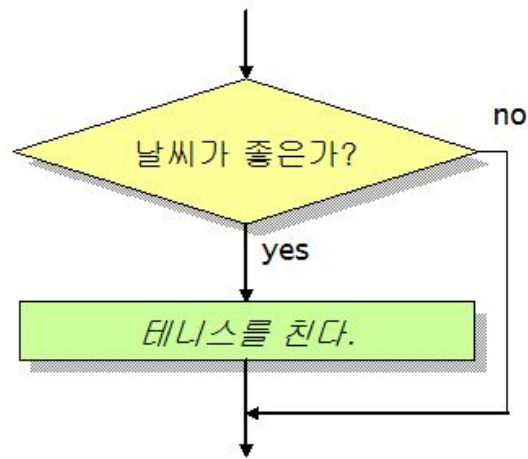
제어문



조건문의 종류

- if 문 , if...else 문, if...else if 문
- switch 문

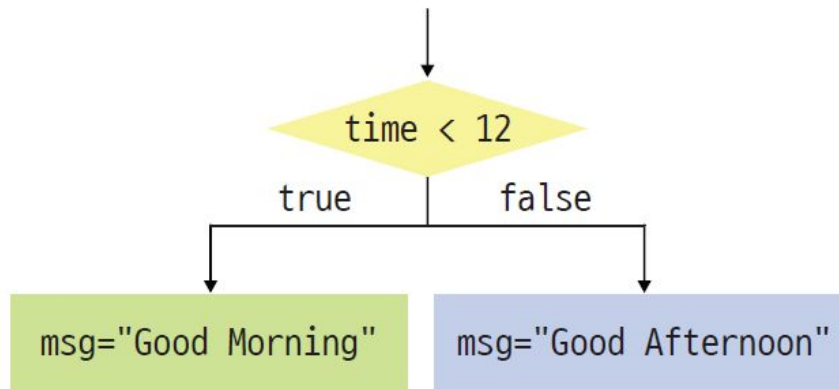
if 문



```
if (time<12)
{
    greeting="Good Morning!";
}
```

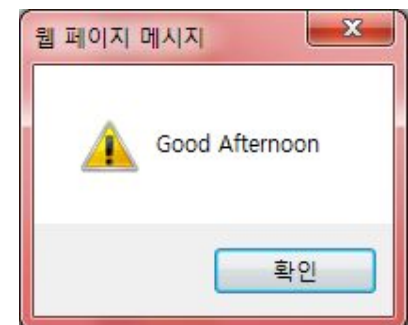
if-else 문

```
if (time<12)
{
    msg="Good Morning!";
}
else{
    msg="Good Afternoon!";
}
```



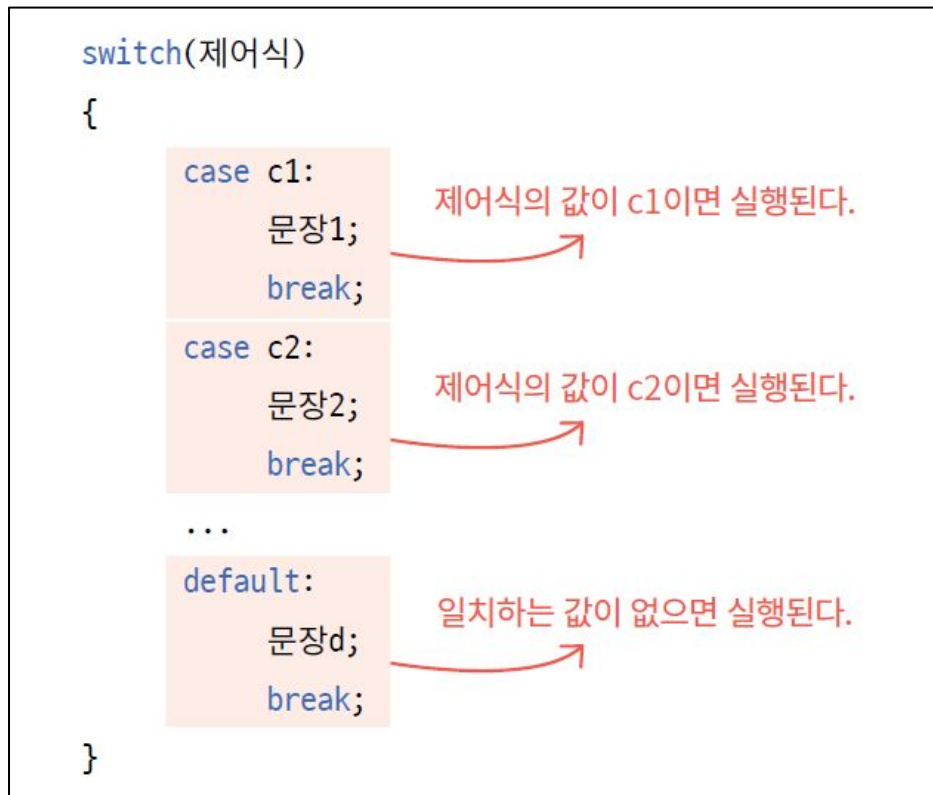
연속적인 if 문

```
<script>
  var msg = "";
  var time = new Date().getHours();
  if (time < 12) {                      // 12시 이전이면
    msg = "Good Morning";
  }
  else if (time < 18) {                 // 오후 6시 이전이면
    msg = "Good Afternoon";
  }
  else {                               // 그렇지 않으면(오후 6시 이후이면)
    msg = "Good evening";
  }
  alert(msg);
</script>
```



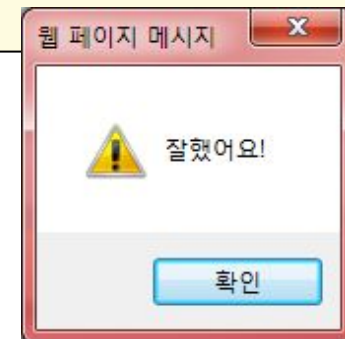
switch 문

- 만약 가능한 실행 경로가 여러 개인 경우에는 **switch** 문을 사용하는 것이 좋다. **switch** 문은 제어식의 값에 따라 다음에 실행할 문장을 결정하게 된다.



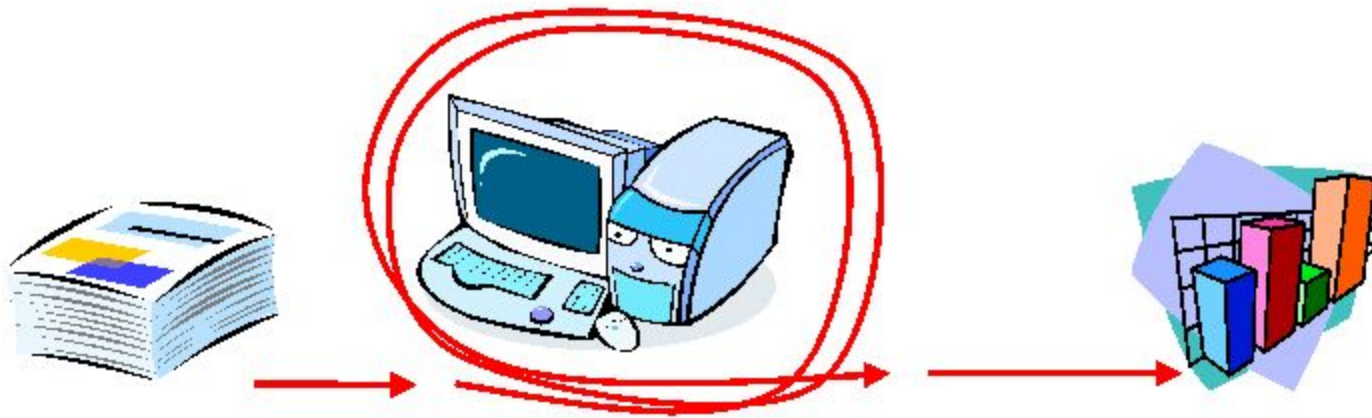
switch 문

```
<script>
  var grade = prompt("성적을 입력하시오:", "A-F사이의 문자로");
  switch (grade) {
    case 'A': alert("잘했어요!");
              break;
    case 'B': alert("좋은 점수군요");
              break;
    case 'C': alert("괜찮은 점수군요");
              break;
    case 'D': alert("좀더 노력하세요");
              break;
    case 'F': alert("다음학기 수강하세요");
              break;
    default: alert("알수없는 학점입니다.")
  }
</script>
```



반복문

- 같은 처리 과정을 여러 번 되풀이하는 것



반복문의 종류

- **while** – 지정된 조건이 참이면 반복 실행한다.
- **for** – 정해진 횟수 동안 코드를 반복 실행한다.

while 문

```
var i = 0;
```

```
while( i < 10 ) ←반복 조건, 변수 i가 10보다 작으면 반복 계속
```

```
{
```

```
    document.write(i+" <br>"); ←반복되는 문장, i를 화면에 출력한다.
```

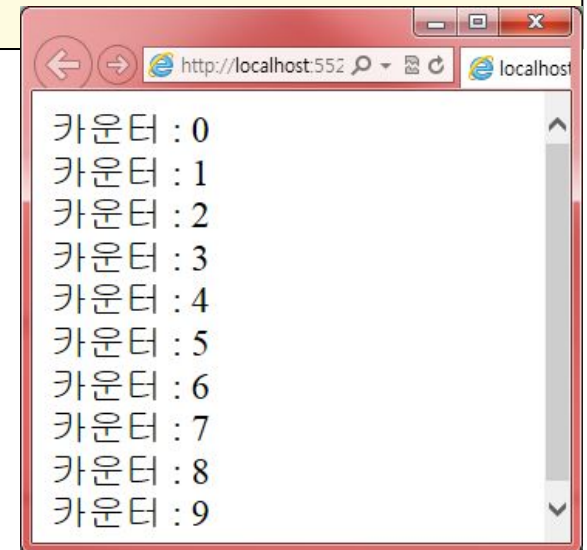
```
    i++; ←한 번의 반복마다 i를 하나 증가시킨다.
```

```
}
```

이 문장이 없으면 무한히 반복한다.

while 문

```
<script>
  var i = 0;
  while (i < 10) {
    document.write("카운터 : " + i + "<br />");
    i++;
  }
</script>
```

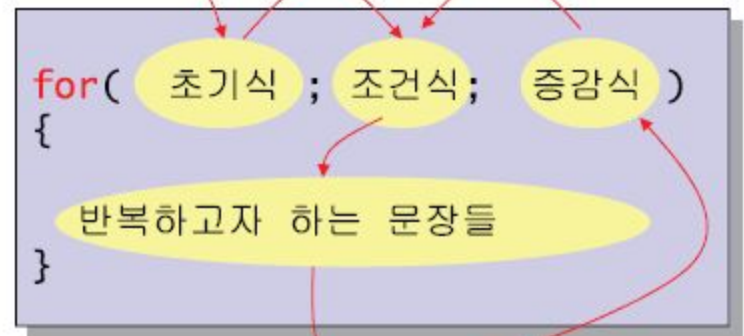


for 문

초기식 조건식 증감식

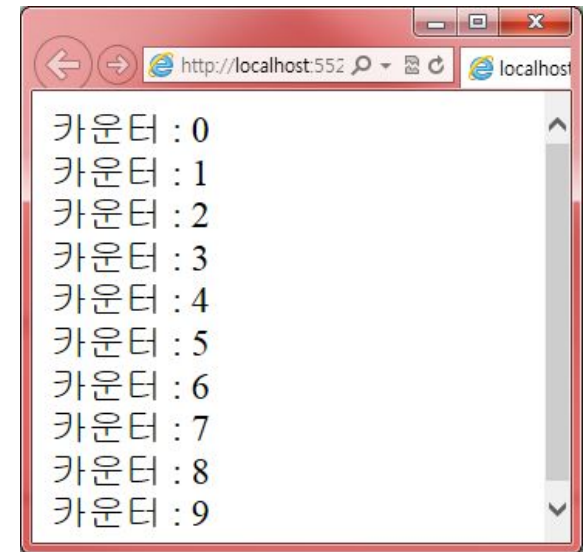
```
for( i=0 ; i<10 ; i++ )  
{  
    document.write(i+"<br />");  
}
```

반복되는 문장



for 문

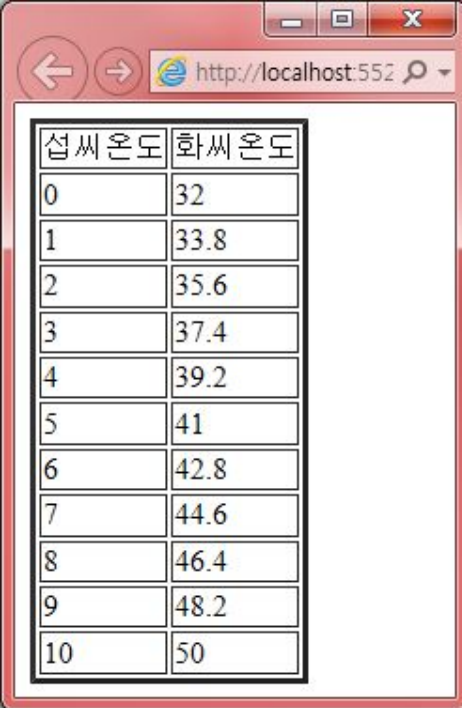
```
<script>
  var i = 0;
  for (i = 0; i < 10; i++) {
    document.write("카운터 : " + i + "<br />");
  }
</script>
```



JS in HTML 예제

```
<html>
<head>
  <title>온도 변환기</title>
</head>

<body>
  <table border="3">
    <tr>
      <td>섭씨 온도</td>
      <td>화씨 온도</td>
    </tr>
    <script>
      for (celsius = 0; celsius <= 10; celsius = celsius + 1) {
        document.write("<tr><td>" + celsius + "</td><td>"
          + ((celsius * 9.0 / 5) + 32) + "</td></tr>");
      }
    </script>
  </table>
</body>
</html>
```



섭씨 온도	화씨 온도
0	32
1	33.8
2	35.6
3	37.4
4	39.2
5	41
6	42.8
7	44.6
8	46.4
9	48.2
10	50

중첩 반복문 예제

```
<script>
```

```
document.write("<h1>구구단표</h1>");  
document.write("<table border=2 width=50%");
```

```
for (var i = 1; i <= 9; i++) {  
    document.write("<tr>");  
    document.write("<td>" + i + "</td>");  
  
    for (var j = 2; j <= 9; j++) {  
        document.write("<td>" + i * j + "</td>");  
    }  
}
```

```
    document.write("</tr>");  
}
```

```
document.write("</table>");
```

```
</script>
```



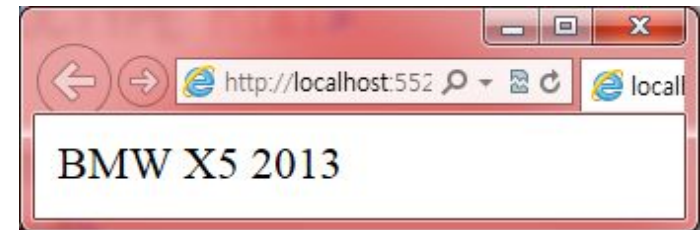
for/in 반복문

```
<script>
  var myCar = { make: "BMW", model: "X5", year: 2013 };
  var txt="";

  for (var x in myCar) {

      txt += myCar[x] + " ";

  }
  document.write(txt);
</script>
```

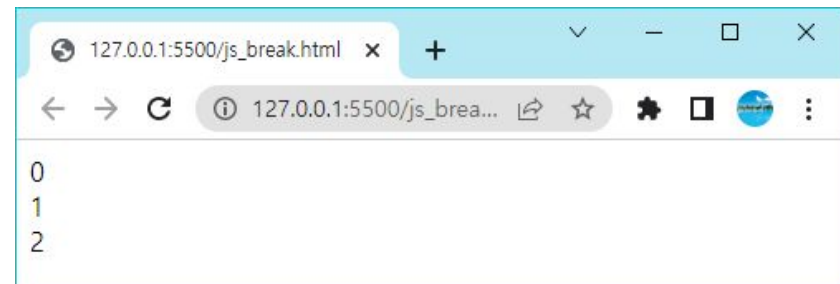


break 문장

- do/while 문은 while 문과 비슷하나 반복 조건을 루프의 처음이 아니라 루프의 끝에서 검사한다는 것이 다르다.

```
<script>
  var msg = "";
  for (let i = 0; i < 10; i++) {

    if (i == 3) break;
    msg += i + "<br>";
  }
  document.write(msg);
</script>
```

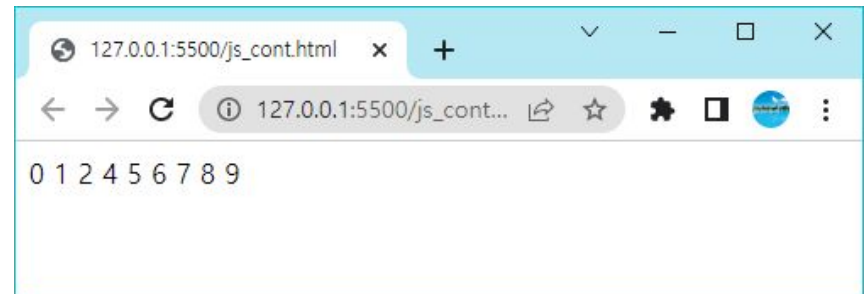


continue 문장

- do/while 문은 while 문과 비슷하나 반복 조건을 루프의 처음이 아니라 루프의 끝에서 검사한다는 것이 다르다.

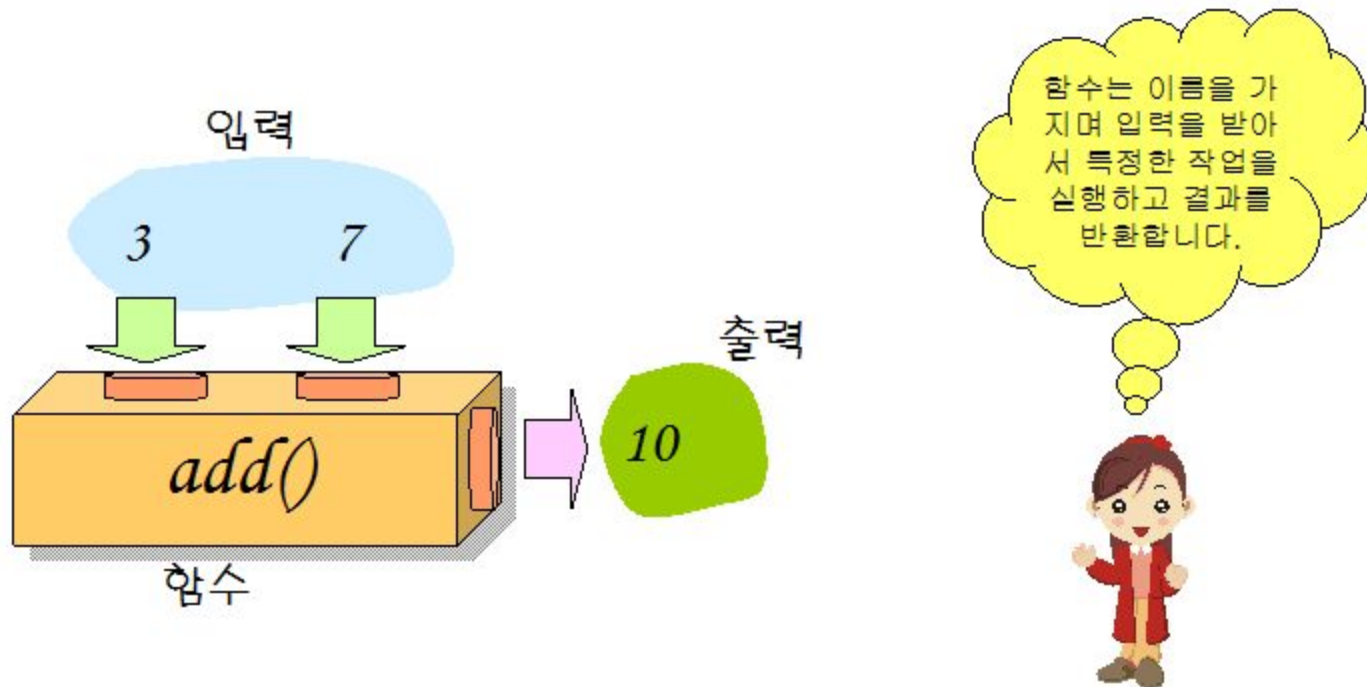
```
<script>
  var msg = "";

  for (let i = 0; i < 10; i++) {
    if (i == 3) continue;
    msg += i + " ";
  }
  document.write(msg);
</script>
```



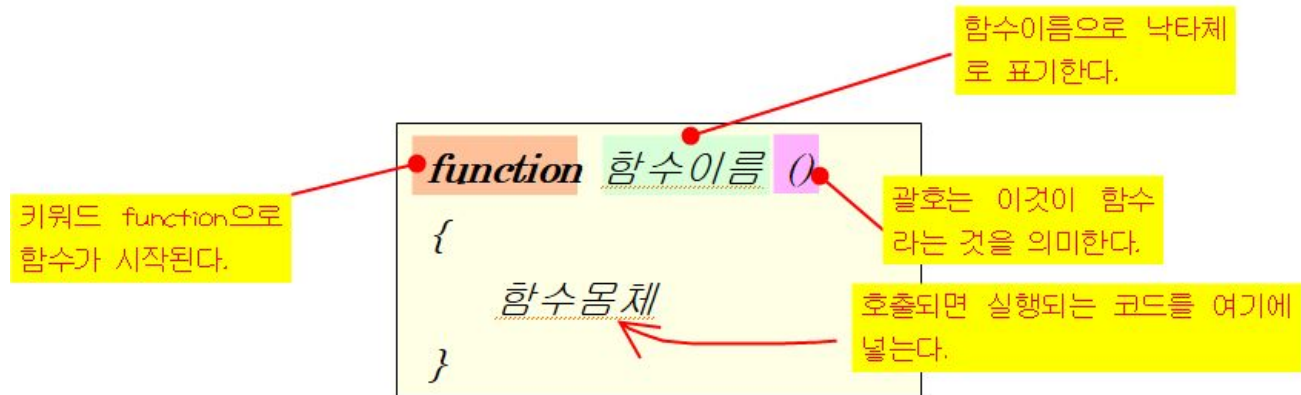
함수

- 함수는 입력을 받아서 특정한 작업을 수행하여서 결과를 반환하는 블랙 박스



함수

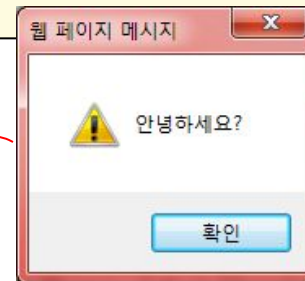
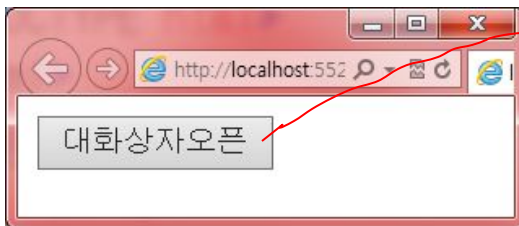
- 함수는 입력을 받아서 특정한 작업을 수행하여서 결과를 반환하는 블랙 박스



예제

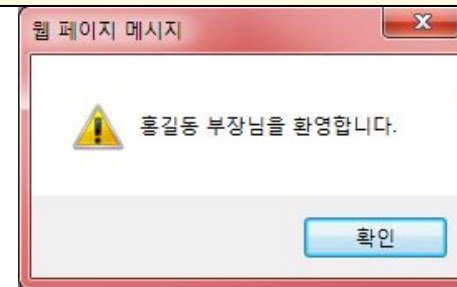
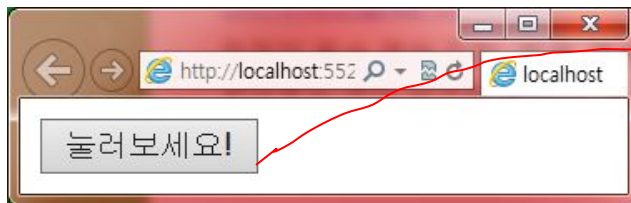
```
<!DOCTYPE html>
<html>
<head>
  <script>
    function showDialog() {
      alert("안녕하세요?");
    }
  </script>
</head>

<body>
  <button onclick="showDialog()">대화상자오픈</button>
</body>
</html>
```



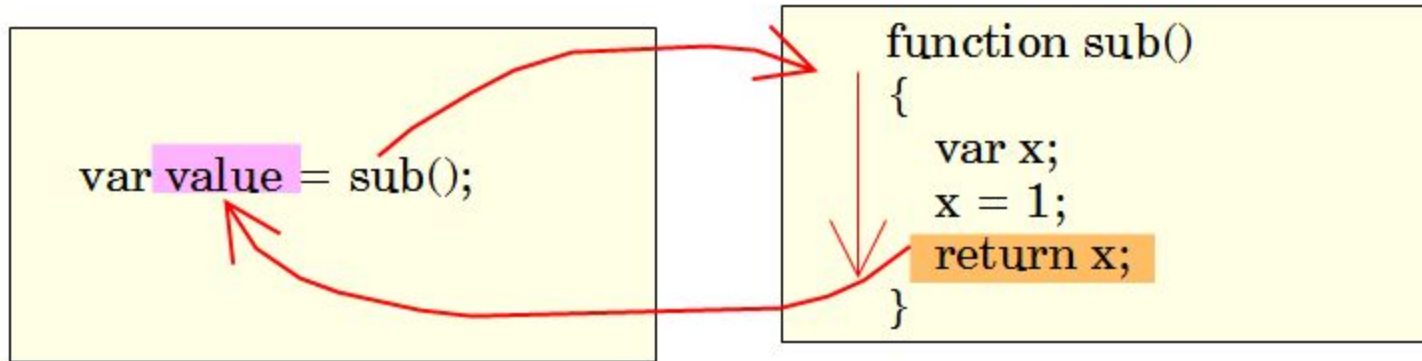
인수와 매개 변수

```
<!DOCTYPE html>
<html>
<head>
  <script>
    function greeting(name, position) {
      alert(name + " " + position + "님을 환영합니다.");
    }
  </script>
</head>
<body>
  <button onclick="greeting('홍길동', '부장')">눌러보세요!</button>
</body>
</html>
```



함수의 반환값

- `return` 문장을 사용하여 외부로 값을 반환



무명(익명)함수

- 자바스크립트에서는 함수 이름을 주지 않고 만들어서 한 번만 사용하는 경우도 많다. 이것을 무명 함수(**anonymous function**)라고 한다.

```
function showDialog() {  
    alert("안녕하세요?");  
}
```

->

```
let greeting = function()  
{  
    alert("안녕하세요?");  
};  
greeting();
```

ES6. 화살표 함수

- 화살표 함수(**arrow function**)는 ES6에 도입되었다. 화살표 함수를 사용하면 더 짧은 함수 구문을 작성할 수 있다. 화살표 함수는 자바 언어의 람다식과 유사하다

```
function sub(a, b){  
  return a + b;  
}
```



```
(a, b) => { return a+b; }  
또는  
(a, b) => return a+b;  
또는  
(a, b) => a + b;
```

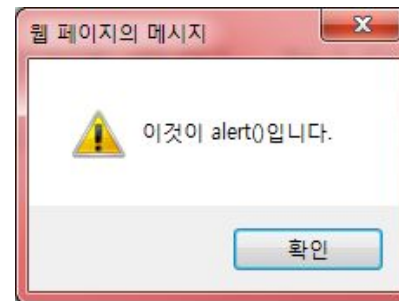
```
function sub(a, b){  
  let data = 29;  
  return a + b + data;  
}
```



```
(a, b) => {  
  let data = 29;  
  return a + b + data;  
}
```

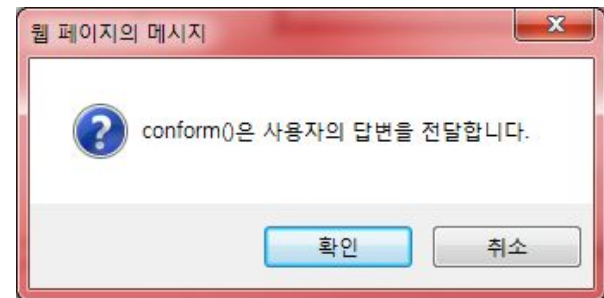
alert() 함수

```
<script>  
  alert("이것이 alert()입니다.");  
</script>
```



confirm() 함수

```
<script>  
  var user = confirm("confirm()은 사용자의 답변을 전달합니다.");  
</script>
```



prompt() 함수

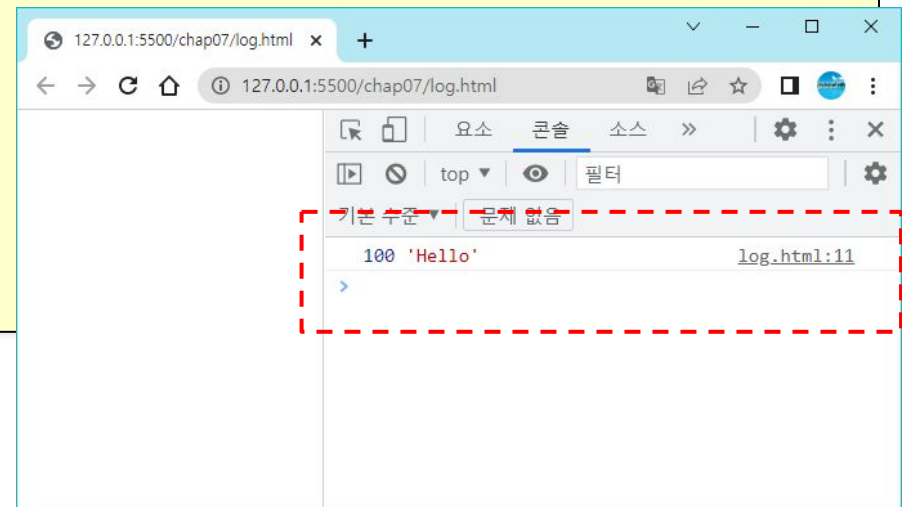
```
<script>  
  var age = prompt("나이를 입력하세요", "만나이로 입력합니다.");  
</script>
```



console.log()

- 자바스크립트 코드를 디버깅할 때 많이 사용하는 함수가 `console.log()`이다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
</head>
<body>
  <script>
    let x = 100;
    let y = 'Hello';
    console.log(x, y);
  </script>
</body>
</html>
```



Q & A

